



沈阳建筑大学

上机实验报告

课程名称:	编译原理
实验题目:	语义分析
专业班级:	计算机 2102
姓 名:	杨奕轩
学 号:	2104240116
日 期:	2024.5.18

【实验目的】

设计并实现一个 LR 语法分析器，根据文法生成 LR 分析表，实现 LR 分析器的状态转移和语法归约操作，并对输入串进行语法分析。

内容：

- (1) 了解常用的自底向上语法分析方法
- (2) 理解 LR 类分析法的基本原理；掌握 LR 分析中总控程序的实现方法；
- (3) 通过设计、编制、调试一个 LR 分析程序，加深对 LR 类分析方法的理理解，实现对单词序列进行语法检查和结构分析。

【基本原理和上机步骤】

1.基本原理

采用自底向上的语法分析方法 LR(k)方法，根据栈中的符号串和向前查看 k 个输入符号，就能唯一确定分析器的动作是移进还是归约，以及用哪个产生式进行归约。

2.上机步骤

LR 分析器的结构：

一个 LR 分析器实际是一个带先进后出存储器（栈）的确定下推自动机，它由一个输入串、一个下推栈和一个带有分析表的总控程序组成。栈中存放着由“历史”和“展望”材料抽象而来的各种“状态”。任何时候，栈顶的状态都代表了整个的历史和已推测出的展望。为了有助于明确归约手续，我们把已归约出的文法符号串也同时放进栈里。

LR 分析器的每一动

作都由栈顶状态和当前输入符号所唯一确定。

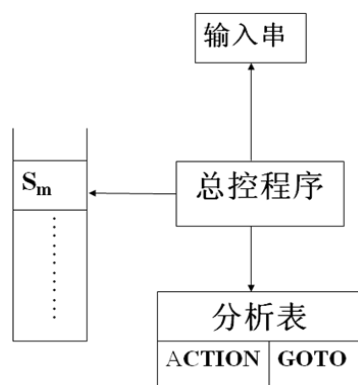


图 2 LR 分析器模型图

分析器的任何一次移动都是根据栈顶状态 S_m 和当前输入符号 a_i ，去查看 ACTION 表并执行 ACTION (S_m, a_i) 规定的动作，直至分析成功或失败。

LR 分析表有两个部分：动作部分 ACTION 和状态转换部分 GOTO。

ACTION[S, a]表明当前状态 S 面临输入符号 a 时应该采取的动作：

- 1、移入：将 S, y 的下一个状态 S 以及当前符号入栈。

- 2、归约：对栈顶的符号串按照某个规则进行归约。
- 3、接受：宣布输入符号串为一个句子。
- 4、报错：宣布输入符号串不是句子。

GOTO[S, U]表示当前状态 S 和非终结符号匹配的时候所转换到的下一个状态。

LR 总控程序：

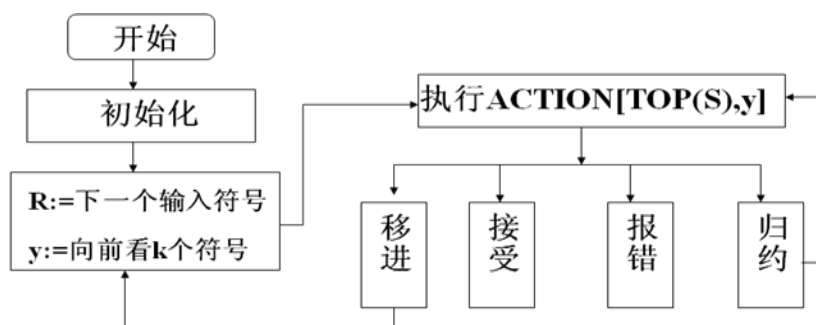


图 3 LR 总控程序示意图

LR 分析器的工作过程是由总控程序根据分析表,使得分析器构型从一种构型向另一种构型变化的过程。初始构型: $(S_0, a_1a_2\cdots a_n \$)$, S_0 为分析器的初态, $\$$ 为输入串的括号。

分析过程的每步结果可表示为: $(S_0X_1S_1\cdots X_mS_m, a_ia_{i+1}\cdots a_n\$)$ 。

分析器的下一次动作是由栈顶状态 S_m 和当前输入符号 a_i 所唯一确定的, 即: 执行 $ACTION[S_m, a_i]$ 规定的动作。经执行各种可能的动作后, 分析器的构型可如下变化:

- 1、若 $ACTION(S_m, a_i) = \text{“移进 } S\text{”}$, 则分析器构型变为 $(S_0X_1S_1\cdots X_mS_ma_iS, a_{i+1}\cdots a_n\$)$ 。
- 2、若 $ACTION(S_m, a_i) = \text{“归约 } A \rightarrow \beta\text{”}$, 则分析器构型变为 $(S_0X_1S_1\cdots X_{m-r}S_{m-r}AS, a_ia_{i+1}\cdots a_n\$)$, 其中 $S = GOTO(S_{m-r}, A)$, $|\beta| = r$ 。
- 3、若 $ACTION(S_m, a_i) = \text{“接受”}$, 则分析成功, 正常停止。
- 4、若 $ACTION(S_m, a_i) = \text{“ERROR”}$, 语法出错, 进行出错处理。

除了这些工作之外, 开始编写代码之前, 我还需要进行 LR 分析表的构造。

【上机结果】

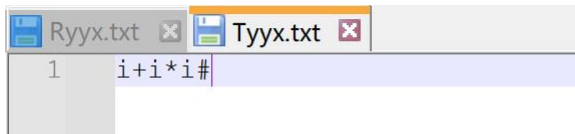
1. 对下列文法, 用 LR (1) 分析法对任意输入的符号串进行分析:

文法

$S \rightarrow E$
 $E \rightarrow E + T$
 $E \rightarrow T$
 $T \rightarrow T * F$
 $T \rightarrow F$
 $F \rightarrow (E)$

F→i

2. 输入文件 Tyyx.txt, 存储输入字符串



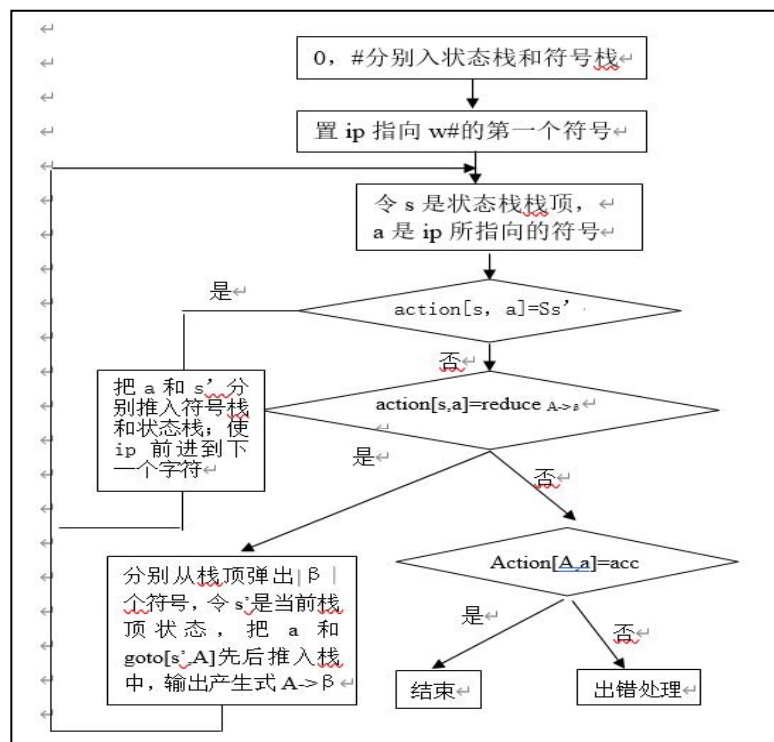
3. 运行结果

保存在 Ryyx.txt 中

	step	状态栈	符号栈	剩余输入串	ACTION	GOTO
1	0)	0	#	i+i*i#	S5	
2	1)	05	#i	+i*i#	r6	3
3	2)	03	#F	+i*i#	r4	2
4	3)	02	#T	+i*i#	r2	1
5	4)	01	#E	+i*i#	S6	
6	5)	016	#E+	i*i#	S5	
7	6)	0165	#E+i	*i#	r6	3
8	7)	0163	#E+F	*i#	r4	9
9	8)	0169	#E+T	*i#	S7	
10	9)	01697	#E+T*	i#	S5	
11	10)	016975	#E+T*i	#	r6	10
12	11)	0169710	#E+T*F	#	r3	9
13	12)	0169	#E+T	#	r1	1
14	13)	01	#E	#	Acc	
15						
16						

【讨论与分析】

实验分析过程



实验过程记录：在实验的过程中，我遇到很多问题，首先我自己通过调试代码的方式解决，如果没有解决，就会去上网查询资料。

实验总结：通过本次 LR 类语法分析的实验，我真切感受到了自底向上分析法的卓越效能与深度魅力。在实践中，通过构建项目集与识别活前缀的逐步推进，我掌握了自动构建 LR 分析表的技巧，这让编译原理中抽象的语法分析概念在我心中变得生动且具象。实验过程中，我直面了状态转换的复杂性与冲突处理的挑战。此外，实验还赋予了我更丰富的编程实践经验，使我得以在算法设计与数据结构应用层面实现了理论与实践的深度融合。这一切不仅显著提升了我的技术实力，还让我对编译器内部运作机制的认知达到了新的高度。这次实验经历是对我个人能力的一次全方位提升，它不仅巩固了我的专业知识，更激发了我对编译器设计与优化的浓厚兴趣。

【程序源代码】

```
#include <fstream>
#include <iostream>
#include <stdlib.h>
#include <string>
struct code_val {
    char code;
    char val[20];
    char LR0[50][50][100];
};
char LR0[50][50][100] = {{"null", "null", "null",
    "null", "S4" , "S5" , "null", "null", "1" , "2" , "3" },// 0
    {"S6" , "S7" , "null", "null", "null",
    "null", "null", "acc" , "null", "null", "null"},// 1
    {"r3" , "r3" , "S8" ,
    "S9" , "null", "null", "r3" , "r3" , "null", "null", "null"},// 2
    {"r6" , "r6" , "r6" ,
    "r6" , "null", "null", "null"},// 3
    {"r8" , "r8" , "r8" ,
    "r8" , "null", "null", "null"},// 4
    {"null", "null", "null",
    "null", "S4" , "S5" , "null", "null", "10" , "2" , "3" },// 5
    {"null", "null", "null",
    "null", "S4" , "S5" , "null", "null", "null", "11" , "3" },// 6
    {"null", "null", "null",
    "null", "S4" , "S5" , "null", "null", "null", "12" , "3" },// 7
    {"null", "null", "null",
    "null", "S4" , "S5" , "null", "null", "null", "null", "13" },// 8
    {"null", "null", "null",
    "null", "S4" , "S5" , "null", "null", "null", "null", "14" },// 9
    {"S6" , "S7" , "null",
    "null", "null", "null", "S15" , "null", "null", "null", "null"},// 10
```

```

        {"r1" , "r1" , "S8" ,
"S9" , "null" , "null" , "r1" , "r1" , "null" , "null" , "null"},// 11
        {"r2" , "r2" , "S8" ,
"S9" , "null" , "null" , "r2" , "r2" , "null" , "null" , "null"},// 12
        {"r4" , "r4" , "r4" ,
"r4" , "null" , "null" , "null"},// 13
        {"r5" , "r5" , "r5" ,
"r5" , "null" , "null" , "null"},// 14
        {"r7" , "r7" , "r7" ,
"r7" , "null" , "null" , "null"},// 15
    };

const char* p[] = { //产生式
"S→E", "E→E+T", "E→T", "T→T*F", "T→F", "F→(E)", "F→i"
};

const char TNT[] = "+*()i#ETF"; //LR分析表的字符
const int M[][9] = { // LR分析表数字化, 列字符 + *()i#ETF用数字012345678标识。
{ 0, 0, 4, 0, 5, 0, 1, 2, 3}, //0表示出错, s4用4表示。
{ 6, 0, 0, 0, 0, 99}, //Acc用99表示
{-2, 7, 0, -2, 0, -2}, //r2用-2表示
{-4, -4, 0, -4, 0, -4},
{ 0, 0, 4, 0, 5, 0, 8, 2, 3},
{-6, -6, 0, -6, 0, -6},
{ 0, 0, 4, 0, 5, 0, 0, 9, 3},
{ 0, 0, 4, 0, 5, 0, 0, 0, 10},
{ 6, 0, 0, 11},
{-1, 7, 0, -1, 0, -1},
{-3, -3, 0, -3, 0, -3},
{-5, -5, 0, -5, 0, -5}
};

int col(char); //列定位函数原型
using namespace std;
void main()
{
    int state[50] = { 0 }; //状态栈初值
    char symbol[50] = { '#' }; //符号栈初值
    int top = 0; //栈顶指针初值
    ofstream cout("Ryyx.txt"); //语法分析结果输出至文件result.txt
    ifstream cin("Tyyx.txt"); // 从test.txt中输入词法分析结果
    struct code_val t; //结构变量, 存放单词二元式。
    //t.val = "i+i*i#";
    cin >> t.code >> t.val; //读一单词
    int action;
    int i, j = 0; //输出时使用的计数器, 并非必要。
    cout << "step" << '\t' << "状态栈" << '\t' << "符号栈" << '\t' << "剩余输入串" << '\t'

```

```

<<"ACTION"
    << '\t' << "GOTO" << endl; //输出标题并非必要。
    do {
        cout << j++ << ')' << '\t'; //输出step, 并非必要。
        for (i = 0; i <= top; i++) cout << state[i]; cout << '\t'; //输出状态栈内容, 并非必要。
        for (i = 0; i <= top; i++) cout << symbol[i]; //输出符号栈内容, 并非必要。
        cout << '\t' << t.code << endl; //输出当前输入符号(单词种别), 并非必要。
        action = M[state[top]][col(t.code)];
        if (action > 0 && action != 99) { //移进
            state[++top] = action;
            symbol[top] = t.code;
            t.code = t.val[0];
            for (int x = 0; x < 20; x++) {
                t.val[x] = t.val[x + 1];
            }
            //cin >> t.code >> t.val; //读一单词
        }
        else if (action < 0) { //归约
            if (strcmp(p[-action] + 3, "ε")) //ε产生式的右部符号串长度为0, 无需退栈。
                top = top - (strlen(p[-action]) - 3); // "→"为汉字, 占二字节, 故减3。
            state[top + 1] = M[state[top]][col(p[-action][0])]; //产生式左部符号
            symbol[++top] = p[-action][0];
        }
        else if (action == 99) { //接受
            cout << '\t' << "Acc" << endl;
            break;
        }
        else { //出错
            cout << "Err in main()>" << action << endl;
            break;
        }
    } while (1);
}

int col(char c) //将字符+*()#ETF分别转换为数字012345678
{
    for (int i = 0; i < (int)strlen(TNT); i++)
        if (c == TNT[i]) return i;
    cout << "Err in col char>" << c << endl;
    exit(0); //终止程序运行
}

```